

UNIVERSIDAD DEL VALLE
Facultad de Ingeniería
Escuela de Ingeniería de Sistemas y Computación
Análisis de Algoritmos II

Informe de Proyecto

Proyecto de curso

Calculando el plan de riego óptimo de una finca

Estudiantes:	Juan Carlos Cruz Muñoz (1824389), David Alejandro Enciso Gutierrez (), Juan Esteban Rodriguez Valencia (), Estiven Andres Martinez Granados ()
Profesores:	Jesús Alexander Aranda
Monitor:	Samuel Galindo

Cali, Colombia
Mayo de 2026

Índice

1. Introducción	2
2. El Problema del Riego Óptimo	2
2.1. Descripción del Problema	2
2.2. Formalización Matemática	3
3. Solución Mediante Fuerza Bruta	4
3.1. Complejidad	4
3.2. Corrección	4
4. Solución Mediante Algoritmo Voraz	4
4.1. Descripción de la Estrategia	4
4.1.1. Correspondencia entre estrategia e implementación	6
4.2. Aplicación del algoritmo y evaluación experimental	8
4.2.1. Aplicación del algoritmo a casos concretos	8
4.2.2. Evaluación experimental	10
4.2.3. Síntesis e interpretación de la evidencia	17
4.3. Complejidad y Corrección	19
5. Solución Mediante Programación Dinámica	21
5.1. Descripción de la Estrategia	21
5.1.1. Correspondencia entre estrategia e implementación	22
5.2. Subestructura Óptima y Corrección	23
5.2.1. Subestructura óptima	23
5.2.2. Definición recursiva	23
5.2.3. Corrección	23
5.3. Aplicación del algoritmo y evaluación experimental	24
5.3.1. Aplicación a los ejemplos del enunciado	24
5.3.2. Evaluación experimental	24
5.4. Análisis de Complejidad	25
6. Comparación de Estrategias	25
7. Conclusiones	26

1. Introducción

Este informe desarrolla el problema del riego óptimo planteado en el enunciado del proyecto de curso. El objetivo es construir y analizar tres enfoques para programar el orden de riego de los tablones de una finca: fuerza bruta, algoritmo voraz y programación dinámica. Cada estrategia se estudia desde tres perspectivas: su idea algorítmica, su complejidad y su capacidad para producir una solución óptima.

La implementación fue realizada en C++ y se encuentra disponible en el repositorio del proyecto: <https://github.com/Juan-Carlos-Cruz/Proyecto-I-ADA-II>. La lógica principal reside en `backend/src/algoritmos.cpp`, donde se implementan las funciones `roFB`, `roV` y `roPD`, correspondientes a fuerza bruta, voraz y programación dinámica. El proyecto también incluye una interfaz para cargar archivos de entrada, ejecutar los algoritmos y revisar las salidas obtenidas.

La dificultad principal del problema está en que el costo no solo depende de qué tablón se riega, sino también del instante exacto en que comienza su riego. Esto hace que el orden completo de la programación sea determinante. Como consecuencia, una decisión aparentemente buena en el corto plazo puede afectar negativamente el costo global. Por eso resulta pertinente comparar una estrategia exhaustiva, una heurística voraz y una formulación exacta mediante programación dinámica.

2. El Problema del Riego Óptimo

2.1. Descripción del Problema

Una finca está compuesta por varios tablones y se dispone de un único recurso móvil de riego. Cada tablón tiene cuatro atributos relevantes:

- **Tiempo de supervivencia** (ts_i): número máximo de días que puede pasar sin ser regado antes de sufrir afectación grave.
- **Tiempo de riego** (tr_i): días requeridos para regar completamente el tablón.
- **Prioridad** (p_i): nivel de importancia del tablón, entre 1 y 4.
- **Tiempo de riego perfecto** (rp_i): instante ideal para iniciar el riego del tablón.

Como solo existe un recurso de riego, en cada instante se debe decidir qué tablón atender primero. El problema consiste en hallar un orden de riego que minimice el costo total de afectación de los cultivos. La función de costo del enunciado penaliza tres situaciones:

- si el tablón se riega exactamente en su día perfecto, se obtiene el costo más bajo;
- si se riega antes de superar su límite de supervivencia, pero no en el instante perfecto, el costo es moderado;
- si se riega tarde, la penalización aumenta y además se multiplica por la prioridad del tablón.

En términos prácticos, el problema es una variante de secuenciamiento de tareas con una función objetivo dependiente del tiempo de inicio. Esto significa que el costo de regar un tablón no es fijo, sino que varía según el momento en que se atiende. Por ello, no basta con priorizar los tablonos más urgentes: en algunos casos conviene postergar el riego de un tablón para acercarse a su día perfecto y así reducir su costo, o adelantarlo si esperar incrementa la penalización. En consecuencia, la solución óptima exige evaluar cómo el orden de programación de riego de cada tablón afecta el costo total de la secuencia.

2.2. Formalización Matemática

Sea una finca

$$F = \langle T_0, T_1, \dots, T_{n-1} \rangle,$$

donde cada tablón se define como

$$T_i = \langle ts_i, tr_i, p_i, rp_i \rangle,$$

con $0 \leq rp_i \leq ts_i - tr_i$.

Una programación de riego es una permutación

$$\Pi = \langle \pi_0, \pi_1, \dots, \pi_{n-1} \rangle$$

de los índices $\{0, 1, \dots, n-1\}$, que indica el orden en el que se riegan los tablonos.

El instante de inicio del riego de cada tablón depende de los tablonos programados antes que él. Si t_i^Π representa el instante en que empieza a regarse el tablón i bajo la programación Π , entonces:

$$t_{\pi_0}^\Pi = 0, \quad t_{\pi_j}^\Pi = \sum_{k=0}^{j-1} tr_{\pi_k} \quad \text{para } j \geq 1.$$

La contribución al costo del tablón i al empezar a regarse en el instante t_i^Π es:

$$CR_F^\Pi[i] = \begin{cases} ts_i - (t_i^\Pi + tr_i), & \text{si } t_i^\Pi = rp_i, \\ 2(ts_i - (t_i^\Pi + tr_i)), & \text{si } t_i^\Pi \neq rp_i \text{ y } t_i^\Pi \leq ts_i - tr_i, \\ 2p_i((t_i^\Pi + tr_i) - ts_i), & \text{en otro caso.} \end{cases}$$

El costo total de la programación es:

$$CR_F^\Pi = \sum_{i=0}^{n-1} CR_F^\Pi[i].$$

El problema del riego óptimo consiste entonces en encontrar una permutación Π^* tal que:

$$\Pi^* = \arg \min_{\Pi} CR_F^\Pi.$$

3. Solución Mediante Fuerza Bruta

La estrategia de fuerza bruta considera todas las posibles permutaciones de los n tablonos. Para cada orden se simula el avance del tiempo, se calcula el costo de cada tablón con la función anterior y finalmente se escoge la permutación de menor costo total.

En la implementación del proyecto, la función `roFB` genera las permutaciones usando la rutina estándar `next_permutation`. Se conserva el mejor orden visto hasta el momento y, al finalizar el recorrido exhaustivo, se vuelve a evaluar ese orden para construir la salida completa con detalles por tablón.

3.1. Complejidad

El número de permutaciones posibles de n tablonos es $n!$. Evaluar una permutación requiere recorrer los n tablonos una vez, por lo que el costo de evaluación es $O(n)$. En consecuencia, el tiempo total es:

$$O(n! \cdot n).$$

El espacio auxiliar es $O(n)$, ya que solo se almacenan la permutación actual, la mejor permutación encontrada y algunas variables escalares. Sin embargo, aunque el consumo de memoria es moderado, el crecimiento factorial del tiempo vuelve a esta solución inviable para instancias medianas. En la práctica, resulta razonable solo para valores pequeños de n .

3.2. Corrección

La corrección de la fuerza bruta es directa. El conjunto de soluciones factibles del problema es precisamente el conjunto de todas las permutaciones de los n tablonos. Como el algoritmo recorre todas esas permutaciones y calcula exactamente su costo, necesariamente inspecciona también la solución óptima global. Al escoger al final la permutación de menor costo entre todas las evaluadas, el resultado devuelto coincide con la definición del óptimo del problema.

Por tanto, la estrategia de fuerza bruta siempre produce la respuesta correcta.

4. Solución Mediante Algoritmo Voraz

4.1. Descripción de la Estrategia

La implementación voraz del proyecto corresponde a la función `roV`. El algoritmo recibe una finca

$$F = \langle T_0, \dots, T_{n-1} \rangle$$

y construye una permutación Π_V que define el orden de riego. Para cada tablón i calcula el valor

$$\rho_i = \frac{tr_i}{p_i},$$

y luego ordena los tableros de menor a mayor según ρ_i . Si dos tableros empatan en ese valor, se desempata usando el margen

$$ts_i - tr_i,$$

privilegiando primero el tablero con menor margen.

Intuición. Si dos tableros ya están en el caso tardío, el costo de regar i en el instante t queda dominado por

$$2p_i((t + tr_i) - ts_i).$$

Aquí aparecen dos fuerzas: un tr_i grande retrasa más a los demás, y un p_i grande hace más caro ese retraso. Por eso el cociente $\frac{tr_i}{p_i}$ intenta equilibrar duración y penalización.

Comparación local. Para justificar formalmente el criterio voraz, comparemos dos tableros i y j , suponiendo que ambos ya están en caso 3 en el instante t . En ese contexto se evalúan dos únicos órdenes posibles: regar primero i y luego j , o regar primero j y luego i .

Si se riega primero i , el costo conjunto es:

$$C(i \rightarrow j) = 2p_i(t + tr_i - ts_i) + 2p_j(t + tr_i + tr_j - ts_j).$$

Si se riega primero j , el costo conjunto es:

$$C(j \rightarrow i) = 2p_j(t + tr_j - ts_j) + 2p_i(t + tr_j + tr_i - ts_i).$$

Al restar ambos costos se obtiene:

$$\Delta_{ij} = C(i \rightarrow j) - C(j \rightarrow i) = 2(p_j tr_i - p_i tr_j).$$

Para que sea mejor poner i antes que j , esta diferencia debe ser menor o igual que cero:

$$\Delta_{ij} \leq 0.$$

Sustituyendo la expresión anterior:

$$2(p_j tr_i - p_i tr_j) \leq 0.$$

De ahí se obtiene:

$$p_j tr_i \leq p_i tr_j.$$

Y como las prioridades son positivas, se puede dividir por $p_i p_j$ y concluir que

$$\frac{tr_i}{p_i} \leq \frac{tr_j}{p_j}.$$

Intuición del desempate. Cuando dos tablonos comparten el mismo cociente $\frac{tr}{p}$, se desempata por la holgura $ts_i - tr_i$: el umbral de tiempo de inicio de riego a partir del cual el tablón entra al caso 3. El tablón con menor holgura tiene ese umbral más bajo, por lo que es más fácil que cualquier espera lo lleve al caso tardío e incrementa su costo. Por eso se atiende primero. Este criterio no se demuestra óptimo formalmente, pero es coherente con la función de costo del problema.

Desde el punto de vista computacional, la estrategia tiene tres pasos:

Paso	Operación	Complejidad
1	Calcular $\frac{tr_i}{p_i}$ y $ts_i - tr_i$ para cada tablón	$O(n)$
2	Ordenar por $\frac{tr}{p}$ y desempatar por $ts - tr$	$O(n \log n)$
3	Simular la permutación y calcular los costos	$O(n)$

Tabla 1: Desglose operacional del algoritmo voraz.

4.1.1. Correspondencia entre estrategia e implementación

El siguiente fragmento no se incluye para deducir la complejidad línea por línea, sino para mostrar que la implementación en C++ preserva exactamente el criterio voraz descrito antes. La implementación de `roV` no reordena directamente los tablonos, sino sus índices. Para simplificar la escritura, definamos

$$\rho_i = \frac{tr_i}{p_i}, \quad h_i = ts_i - tr_i.$$

Entonces, para dos tablonos i y j , el comparador usado por `sort` aplica estas reglas en orden:

1. Si $\rho_i < \rho_j$, entonces i va antes que j .
2. Si $\rho_i = \rho_j$, va primero el tablón con menor holgura, es decir, el que tenga menor h .
3. Si también empatan en holgura, va primero el de menor índice original.

Como el código evita divisiones en punto flotante, la comparación principal no se implementa calculando cocientes decimales, sino mediante producto cruzado:

$$\frac{tr_i}{p_i} < \frac{tr_j}{p_j} \iff tr_i p_j < tr_j p_i.$$

El siguiente fragmento muestra el núcleo de esa implementación:

```
vector<int> orden(cantidad_tablonos);
iota(orden.begin(), orden.end(), 0);

sort(orden.begin(), orden.end(), [&](int indice_a, int indice_b) {
    const Tablon& a = tablonos[indice_a];
    const Tablon& b = tablonos[indice_b];
```

```

long long ratio_a = 1LL * a.tiempo_riego * b.prioridad;
long long ratio_b = 1LL * b.tiempo_riego * a.prioridad;
if (ratio_a != ratio_b) return ratio_a < ratio_b;

const int margen_a = a.tiempo_supervivencia - a.tiempo_riego;
const int margen_b = b.tiempo_supervivencia - b.tiempo_riego;
if (margen_a != margen_b) return margen_a < margen_b;

return indice_a < indice_b;
});

```

Brevemente, cada bloque del fragmento cumple una función específica dentro de la estrategia voraz:

- `vector<int>orden(cantidad_tablones);` reserva el arreglo donde se guardará la permutación de índices que luego se ordenará.
- `iota(orden.begin(), orden.end(), 0);` inicializa ese arreglo como $\langle 0, 1, \dots, n-1 \rangle$, es decir, con el orden original de los tablones.
- `sort(...)` aplica el comparador voraz sobre esos índices para construir la permutación final.
- `const Tablon& a = tablones[indice_a];` recupera el primer tablán involucrado en la comparación.
- `const Tablon& b = tablones[indice_b];` recupera el segundo tablán involucrado en la comparación.
- `ratio_a` y `ratio_b` implementan el producto cruzado $tr_i p_j$ y $tr_j p_i$, evitando dividir en punto flotante.
- `if (ratio_a != ratio_b) return ratio_a < ratio_b;` decide el orden principal según la regla $\frac{tr_i}{p_i} < \frac{tr_j}{p_j}$.
- `margen_a` y `margen_b` calculan la holgura $ts - tr$ de cada tablán para usarla como segundo criterio.
- `if (margen_a != margen_b) return margen_a < margen_b;` da prioridad al tablán con menor margen cuando la razón principal empata.
- `return indice_a < indice_b;` resuelve el empate final conservando el orden por índice y hace determinista la salida del algoritmo.

Ejemplo. Si T_i tiene $(tr_i, p_i) = (2, 4)$ y T_j tiene $(tr_j, p_j) = (3, 2)$, el código no compara $2/4$ con $3/2$, sino

$$2 \cdot 2 = 4 \quad \text{y} \quad 3 \cdot 4 = 12.$$

Como $4 < 12$, decide $i \prec j$. Si dos tablones empatan en $\frac{tr}{p}$, por ejemplo $(ts_i, tr_i, p_i) = (6, 2, 2)$ y $(ts_j, tr_j, p_j) = (9, 3, 3)$, entonces ambos tienen razón 1 y el desempate pasa a la holgura:

$$ts_i - tr_i = 4 \quad < \quad ts_j - tr_j = 6.$$

Por eso el algoritmo coloca primero a i .

En conjunto, el fragmento confirma que la implementación concreta preserva exactamente la estructura del algoritmo descrito antes: construir una permutación de índices, ordenarla según el criterio $\frac{tr}{p}$, desempatar por holgura y resolver los empates finales de forma determinista por índice original.

4.2. Aplicación del algoritmo y evaluación experimental

4.2.1. Aplicación del algoritmo a casos concretos

Aplicación a los ejemplos del enunciado. Para las dos fincas de la Sección 2.3 del enunciado, el criterio voraz produce la misma permutación:

$$\Pi_V = \langle 0, 1, 3, 2, 4 \rangle.$$

Ejemplo 1. Para

$$F_1 = \langle \langle 10, 3, 4, 0 \rangle, \langle 6, 3, 3, 1 \rangle, \langle 2, 2, 1, 0 \rangle, \langle 8, 1, 1, 6 \rangle, \langle 10, 4, 2, 5 \rangle \rangle$$

los valores relevantes son:

Tablón	ts	tr	p	rp	$\frac{tr}{p}$	$ts - tr$
T_0	10	3	4	0	0.75	7
T_1	6	3	3	1	1.00	3
T_2	2	2	1	0	2.00	0
T_3	8	1	1	6	1.00	7
T_4	10	4	2	5	2.00	6

Tabla 2: Valores usados por el voraz en la finca F_1 .

El orden voraz es

$$T_0 \rightarrow T_1 \rightarrow T_3 \rightarrow T_2 \rightarrow T_4.$$

Al simular esa programación se obtiene:

Paso	Tablón	t_{inicio}	Caso aplicado	Costo
1	T_0	0	caso 1: $10 - (0 + 3)$	7
2	T_1	3	caso 2: $2(6 - (3 + 3))$	0
3	T_3	6	caso 1: $8 - (6 + 1)$	1
4	T_2	7	caso 3: $2 \cdot 1 \cdot ((7 + 2) - 2)$	14
5	T_4	9	caso 3: $2 \cdot 2 \cdot ((9 + 4) - 10)$	12

Tabla 3: Evaluación detallada del voraz sobre F_1 .

$$CR_{F_1}^{\Pi_V} = 7 + 0 + 1 + 14 + 12 = 34.$$

La programación óptima por fuerza bruta es

$$\Pi^* = \langle 2, 1, 0, 3, 4 \rangle$$

con costo 20. La diferencia se explica porque el voraz posterga T_2 , que tiene holgura cero ($ts_2 - tr_2 = 0$), y lo condena a caer en caso 3.

Por tanto, la solución voraz para F_1 **no es óptima**.

Ejemplo 2. Para

$$F_2 = \langle \langle 9, 3, 4, 0 \rangle, \langle 5, 3, 3, 2 \rangle, \langle 2, 2, 1, 0 \rangle, \langle 8, 1, 1, 6 \rangle, \langle 6, 4, 2, 1 \rangle \rangle$$

se obtiene la misma permutación voraz. La evaluación queda:

Paso	Tablón	t_{inicio}	Caso aplicado	Costo
1	T_0	0	caso 1: $9 - (0 + 3)$	6
2	T_1	3	caso 3: $2 \cdot 3 \cdot ((3 + 3) - 5)$	6
3	T_3	6	caso 1: $8 - (6 + 1)$	1
4	T_2	7	caso 3: $2 \cdot 1 \cdot ((7 + 2) - 2)$	14
5	T_4	9	caso 3: $2 \cdot 2 \cdot ((9 + 4) - 6)$	28

Tabla 4: Evaluación detallada del voraz sobre F_2 .

$$CR_{F_2}^{\Pi_V} = 6 + 6 + 1 + 14 + 28 = 55.$$

El óptimo es nuevamente

$$\Pi^* = \langle 2, 1, 0, 3, 4 \rangle$$

con costo 32. El patrón de falla es el mismo: un tablón con margen nulo queda demasiado atrás por tener una prioridad baja y, por tanto, un cociente $\frac{tr}{p}$ alto.

Por tanto, la solución voraz para F_2 **no es óptima**.

Para complementar los dos ejemplos del enunciado sin reutilizar la batería de pruebas del repositorio, se diseñaron cuatro fincas adicionales pequeñas:

Ejemplo 3. Para

$$F_3 = \langle \langle 11, 3, 3, 3 \rangle, \langle 13, 3, 3, 2 \rangle, \langle 6, 3, 1, 0 \rangle, \langle 5, 2, 2, 2 \rangle, \langle 5, 4, 4, 1 \rangle \rangle$$

el voraz produce

$$\Pi_V = \langle 4, 3, 0, 1, 2 \rangle \quad \text{con} \quad CR_{F_3}^{\Pi_V} = 30.$$

La salida óptima de referencia es

$$\Pi^* = \langle 4, 3, 0, 1, 2 \rangle \quad \text{con costo} \quad 30.$$

Por tanto, la solución voraz para F_3 **sí es óptima**.

Ejemplo 4. Para

$$F_4 = \langle \langle 5, 4, 4, 0 \rangle, \langle 13, 4, 2, 6 \rangle, \langle 8, 7, 2, 0 \rangle, \langle 9, 1, 1, 1 \rangle, \langle 8, 2, 2, 5 \rangle \rangle$$

el voraz produce

$$\Pi_V = \langle 0, 4, 3, 1, 2 \rangle \quad \text{con} \quad CR_{F_4}^{\Pi_V} = 53.$$

La salida óptima de referencia es

$$\Pi^* = \langle 0, 4, 1, 3, 2 \rangle \quad \text{con costo} \quad 52.$$

Por tanto, la solución voraz para F_4 **no es óptima**.

Ejemplo 5. Para

$$F_5 = \langle \langle 11, 4, 3, 0 \rangle, \langle 8, 5, 4, 2 \rangle, \langle 6, 2, 3, 0 \rangle, \langle 15, 9, 2, 1 \rangle, \langle 5, 1, 4, 0 \rangle \rangle$$

el voraz produce

$$\Pi_V = \langle 4, 2, 1, 0, 3 \rangle \quad \text{con} \quad CR_{F_5}^{\Pi_V} = 40.$$

La salida óptima de referencia es

$$\Pi^* = \langle 2, 4, 1, 0, 3 \rangle \quad \text{con costo} \quad 38.$$

Por tanto, la solución voraz para F_5 **no es óptima**.

Ejemplo 6. Para

$$F_6 = \langle \langle 12, 6, 4, 5 \rangle, \langle 7, 2, 2, 5 \rangle, \langle 9, 4, 3, 2 \rangle, \langle 8, 8, 4, 0 \rangle, \langle 13, 6, 4, 3 \rangle \rangle$$

el voraz produce

$$\Pi_V = \langle 1, 2, 0, 4, 3 \rangle \quad \text{con} \quad CR_{F_6}^{\Pi_V} = 197.$$

La salida óptima de referencia es

$$\Pi^* = \langle 2, 1, 0, 4, 3 \rangle \quad \text{con costo} \quad 196.$$

Por tanto, la solución voraz para F_6 **no es óptima**.

Caso adicional	n	Costo voraz	Costo óptimo	Diferencia	¿Es óptima?
Ejemplo 3	5	30	30	0	Sí
Ejemplo 4	5	53	52	1	No
Ejemplo 5	5	40	38	2	No
Ejemplo 6	5	197	196	1	No

Tabla 5: Resumen de cuatro entradas adicionales diseñadas para el análisis del algoritmo voraz.

4.2.2. Evaluación experimental

Experimento 1: batería de pruebas del repositorio. Para complementar el análisis, aquí se reporta el conjunto completo de los 18 casos de la batería de pruebas del repositorio que sí cuentan con entrada y salida de referencia. Mostrar todos los casos permite respaldar

directamente las estadísticas agregadas y ver cómo varía el desempeño del voraz según el tamaño y la estructura de cada finca.

Test	n	Costo óptimo	Costo voraz	Diferencia	% exceso	¿Voraz = óptimo?
1	5	44	59	15	34.09	No
2	5	126	128	2	1.59	No
3	5	38	57	19	50.00	No
4	5	26	52	26	100.00	No
5	6	193	193	0	0.00	Sí
6	6	256	288	32	12.50	No
7	7	67	80	13	19.40	No
8	7	294	302	8	2.72	No
9	8	143	178	35	24.48	No
10	8	522	534	12	2.30	No
11	10	698	762	64	9.17	No
12	10	526	638	112	21.29	No
13	15	1276	1297	21	1.65	No
14	15	2460	2468	8	0.33	No
15	15	1030	1080	50	4.85	No
16	20	3758	3816	58	1.54	No
17	20	2861	2894	33	1.15	No
18	20	1512	1590	78	5.16	No

Tabla 6: Comparación entre costo óptimo y costo voraz en los 18 casos del repositorio con salida de referencia.

Elemento	Valor / fórmula	Lectura rápida
Casos evaluados	18	Tests con entrada y salida de referencia.
Coincidencias exactas	$1/18 = 5,56\%$	Veces en que el voraz iguala al óptimo.
Diferencia promedio	32,56	Promedio de d_k .
Exceso relativo promedio	16,23%	Promedio de e_k .
Error absoluto por caso	$d_k = C_V^{(k)} - C^{*(k)} $	Distancia entre el costo voraz y el costo óptimo del caso k .
Error relativo por caso	$e_k = 100 \cdot \frac{d_k}{C^{*(k)}}$	Ese mismo error, pero expresado como porcentaje del óptimo.

Tabla 5.1. Resumen estadístico y fórmulas usadas para interpretar la Tabla 5.

Aquí $C_V^{(k)}$ es el costo obtenido por el voraz en el caso k y $C^{*(k)}$ es el costo óptimo de ese mismo caso. En esta batería, el peor caso absoluto fue el **test12** (638 frente a 526) y el peor caso relativo fue el **test4**, donde el voraz duplicó el costo óptimo (52 frente a 26).

Experimento 2: bloque mixto aleatorio de 5000 fincas con $n = 20$. La batería de pruebas permite exhibir contraejemplos y observar con detalle algunos casos concretos, pero no basta para estimar cómo se comporta el voraz de manera típica. Por eso se realiza una validación empírica adicional sobre una muestra grande: la idea es comprobar si los patrones observados antes se mantienen cuando se consideran muchas fincas distintas, obtener promedios más estables y aislar el análisis en un tamaño ya exigente ($n = 20$).

Además de la batería de pruebas, se analizó el bloque `n20` definido por la versión actual del script `generar_tests_aleatorios.py`. En su estado actual, el generador produce únicamente instancias con $n = 20$, con un total de 5000 fincas y semilla fija 20260513. Cada tablón se construye a partir de un perfil aleatorio:

Cómo se genera cada perfil. En todos los perfiles se sigue la misma plantilla:

$$\text{margen} = ts - tr, \quad tr = ts - \text{margen}, \quad rp \in [0, \text{margen}].$$

En el script, esta variable aparece con el nombre `slack`. La diferencia entre perfiles está en cómo se sortean ts , margen y p :

- **Crítico:** se sortea $ts \in [5, 15]$, se fija $\text{margen} = 0$ y se sortea $p \in [1, 4]$. Luego $tr = ts$ y necesariamente $rp = 0$. Intención: modelar tableros que no admiten espera.
- **Urgente:** se sortea $ts \in [5, 10]$, luego $\text{margen} \in [1, \min(3, ts - 1)]$ y $p \in [3, 4]$. Después se calcula $tr = ts - \text{margen}$ y se elige $rp \in [0, \text{margen}]$. Intención: hacer que pocos días de retraso ya cuesten caro.
- **Balanceado:** se sortea $ts \in [7, 13]$, luego $\text{margen} \in [2, \min(6, ts - 1)]$ y $p \in [1, 4]$. Después se calcula $tr = ts - \text{margen}$ y se elige $rp \in [0, \text{margen}]$. Intención: representar un punto medio donde compiten urgencia y día perfecto.
- **Relajado:** se sortea $ts \in [10, 15]$, luego $\text{margen} \in [7, \min(13, ts - 1)]$ y $p \in [1, 4]$. Después se calcula $tr = ts - \text{margen}$ y se elige $rp \in [0, \text{margen}]$. Intención: dar mucho margen para que el día perfecto pese más que la urgencia inmediata.
- **Largo plazo:** se sortea $ts \in [20, 100]$, luego $\text{margen} \in [10, \min(50, ts - 1)]$ y $p \in [1, 4]$. Después se calcula $tr = ts - \text{margen}$ y se elige $rp \in [0, \text{margen}]$. Intención: mezclar horizontes temporales largos con otros más tensos.

Relación con la función de costo. Si t es el instante en que empieza el riego, entonces:

$$t = rp \Rightarrow \text{caso 1}, \quad t \leq \text{margen} \text{ y } t \neq rp \Rightarrow \text{caso 2}, \quad t > \text{margen} \Rightarrow \text{caso 3}.$$

Así, un margen pequeño produce tableros críticos o urgentes; un margen grande deja más espacio para que el día perfecto rp influya. La prioridad p solo agrava el caso 3.

Ejemplos por perfil.

- **Crítico:** $ts = 8$, $\text{margen} = 0$, luego $tr = 8$ y $rp = 0$. Si se inicia en $t = 0$, cae en el caso 1; cualquier retraso lo lleva de inmediato al caso 3.
- **Urgente:** $ts = 9$, $\text{margen} = 2$, luego $tr = 7$ y se puede tomar $rp = 1$. Si $t = 1$, caso 1; si $t = 2$, caso 2; si $t > 2$, caso 3.
- **Balanceado:** $ts = 11$, $\text{margen} = 4$, luego $tr = 7$ y se puede tomar $rp = 2$. Si $t = 2$, caso 1; si $t = 4$, caso 2; si $t > 4$, caso 3.

- **Relajado:** $ts = 14$, $margen = 9$, luego $tr = 5$ y se puede tomar $rp = 6$. Si $t = 6$, caso 1; si $t = 8$, caso 2; si $t > 9$, caso 3.
- **Largo plazo:** $ts = 40$, $margen = 20$, luego $tr = 20$ y se puede tomar $rp = 12$. Si $t = 12$, caso 1; si $t = 18$, caso 2; si $t > 20$, caso 3.

Distribución nominal del generador. Cada tablón se genera de forma independiente con las siguientes probabilidades:

- 10 % crítico;
- 30 % urgente;
- 25 % balanceado;
- 20 % relajado;
- 15 % largo plazo.

Estos porcentajes no vienen impuestos por el enunciado; se eligieron para construir una muestra variada pero no extrema. La idea es que los perfiles tensos dominen ligeramente la mezcla, porque son los que más rápido exponen fallas del voraz, sin eliminar por completo los perfiles con mucha espera, donde el día perfecto rp pesa más.

- el 10 % crítico asegura casos borde, pero evita que toda la muestra quede dominada por instancias demasiado rígidas;
- el 30 % urgente y el 25 % balanceado concentran la mayor parte de la muestra en la zona más informativa, donde compiten urgencia, prioridad y día perfecto;
- el 20 % relajado introduce suficientes tablonos posponibles para que el experimento no favorezca artificialmente al voraz;
- el 15 % largo plazo agrega heterogeneidad temporal sin volver la muestra irrealmente dispersa.

Por qué el generador es válido. La generación siempre preserva las restricciones del enunciado:

$$ts \geq tr, \quad 1 \leq p \leq 4, \quad 0 \leq rp \leq ts - tr.$$

Además, por construcción:

- se cubren simultáneamente casos borde ($ts = tr$), tablonos urgentes, balanceados y relajados;
- no se copian literalmente los casos de la batería de pruebas del proyecto;
- se obtiene una familia sintética coherente con el enunciado y con el estilo de datos observado en la batería real.

Por qué se fijó $n = 20$. Esta decisión tiene dos ventajas metodológicas:

- aísla el comportamiento del criterio voraz en un tamaño ya exigente, sin mezclarlo con el efecto del crecimiento de n ;
- todavía permite resolver todas las instancias por programación dinámica, de modo que cada salida de **roV** se compara contra un óptimo exacto.

Tamaño y validez de la muestra. El bloque usa 5000 fincas de $n = 20$, es decir, 100000 tablones. Ese tamaño basta para que el experimento no dependa de unos pocos casos atípicos y, al mismo tiempo, permite comparar cada salida de **roV** contra un óptimo exacto calculado por programación dinámica.

La mezcla generada también es consistente con el diseño del experimento. En promedio, cada finca contiene 2 tablones críticos, 6 urgentes, 5 balanceados, 4 relajados y 3 de largo plazo. Las frecuencias observadas quedaron prácticamente iguales a las nominales: 9,994 % crítico, 29,858 % urgente, 24,838 % balanceado, 20,239 % relajado y 15,071 % largo plazo, con una diferencia máxima de solo 0,24 puntos porcentuales entre lo esperado y lo observado.

Además, la muestra no está sesgada hacia instancias “fáciles”: el 99,92 % de las fincas contiene al menos un urgente, el 99,54 % al menos un balanceado, el 95,92 % al menos un tablón de largo plazo, y en el 100 % aparece al menos un tablón con margen ≤ 3 y prioridad 3 o 4. Por eso este bloque sirve para evaluar al voraz en un entorno repetible y exigente.

Conclusión metodológica. En síntesis, el experimento es empíricamente sólido porque el generador respeta las restricciones formales, cubre perfiles distintos en proporciones controladas e introduce trampas heurísticas relevantes; además, usa semilla fija y compara siempre contra el costo óptimo real. Bajo esas condiciones, los resultados no dependen de coincidencias fortuitas.

Los resultados del criterio voraz sobre el bloque mixto de $n = 20$, construido con la mezcla de perfiles definida arriba, se resumen en la tabla siguiente:

Métrica	Resultado en 5000 casos con $n = 20$
Coincidencias exactas con el óptimo	219/5000 = 4,38 %
Diferencia absoluta promedio	66.94
Exceso relativo promedio frente al óptimo	1,97 %
Peor diferencia absoluta observada	382

Tabla 7: Resultados del criterio voraz con $n = 20$ sobre el bloque mixto generado a partir de los perfiles definidos.

Resumen estadístico descriptivo. En las 5000 fincas analizadas, el voraz alcanzó exactamente el mismo costo óptimo en 219 casos (4,38 %). Cuando no alcanzó ese costo, la diferencia absoluta promedio fue de 66.94 unidades y el exceso relativo promedio frente

al óptimo fue de apenas 1,97 %. El peor desvío absoluto observado en todo el bloque fue 382.

Interpretación del bloque mixto. En conjunto, esto indica que el voraz rara vez alcanza exactamente el costo óptimo cuando conviven perfiles distintos, aunque el costo adicional que pierde suele mantenerse bajo. Su debilidad principal no está tanto en el valor promedio final, sino en la dificultad para ordenar correctamente tablonos con urgencias, prioridades y días perfectos heterogéneos.

Experimento 3: aislamiento por perfiles. El bloque mixto anterior sirve para medir el comportamiento global del criterio, pero no permite ver qué perfil explica cada error. Por eso, a continuación se realiza un **experimento de aislamiento** por perfil. El manejo del análisis cambia así: en lugar de mezclar tipos de tablonos, se generan cinco bloques separados de 5000 fincas con $n = 20$, cada uno compuesto al 100 % por un solo perfil (crítico, urgente, balanceado, largo plazo o relajado), manteniendo la misma semilla fija 20260513 y comparando siempre contra el óptimo exacto. En cada bloque se reportan las mismas tres medidas: diferencia promedio, exceso relativo promedio y tasa de coincidencia exacta en costo con el óptimo. La tabla siguiente muestra cómo cambia el rendimiento del voraz en esos escenarios puros:

Perfil exclusivo en la Finca ($n = 20$)	Diferencia promedio	Exceso relativo %	Tasa de coincidencia exacta en costo
Fincas 100 % Críticos	0.00	0.00 %	5000/5000 (100 %)
Fincas 100 % Urgentes	2.83	0.05 %	1735/5000 (≈ 35 %)
Fincas 100 % Balanceados	11.13	0.35 %	533/5000 (≈ 11 %)
Fincas 100 % Largo plazo	180.25	1.41 %	25/5000 (≈ 0.5 %)
Fincas 100 % Relajados	62.37	5.53 %	4/5000 (≈ 0.1 %)

Tabla 8: Resultados del experimento de aislamiento por perfil sobre bloques puros de 5000 fincas con $n = 20$.

Resumen estadístico descriptivo. El experimento de aislamiento muestra un contraste nítido entre perfiles. En fincas 100 % críticas, el voraz alcanzó el costo óptimo en los 5000 casos. En fincas 100 % urgentes, todavía mantuvo un desempeño muy alto: diferencia promedio 2.83, exceso relativo 0,05 % y 1735 coincidencias exactas en costo. En cambio, al pasar a perfiles con más espera, esa tasa de costo óptimo cayó a 533/5000 en balanceado, 25/5000 en largo plazo y 4/5000 en relajado.

Interpretación del aislamiento por perfiles. La conclusión es que el voraz funciona muy bien cuando el perfil obliga a decidir rápido, pero pierde confiabilidad cuando crece la holgura. Aun así, esa pérdida no siempre implica un gran daño económico: en balanceado y largo plazo la tasa de costo óptimo cae mucho, pero el exceso relativo sigue siendo moderado. El perfil verdaderamente más adverso es relajado, porque allí el día perfecto pesa más que la urgencia inmediata. En otras palabras, el aislamiento confirma que la principal debilidad del criterio aparece cuando debe ordenar tablonos muy posponibles.

Experimento 4: comparación de criterios voraces. Para justificar inequívocamente la selección de $\frac{tr}{p}$ frente a intuiciones alternativas que a priori podrían parecer razonables (como atender lo más urgente, de mayor prioridad o el día perfecto), se ejecutó un laboratorio comparando todas las 20 combinaciones ordenadas implementadas en `backend/src/voraz_criterios.cpp`: 5 criterios posibles como clave principal y 4 como desempate. El ranking sobre el nuevo conjunto mixto de 5000 instancias fue el siguiente:

Tabla 9: Comparación completa de las 20 combinaciones de criterios voraces sobre el bloque mixto oficial de 5000 fincas con $n = 20$.

Principal	Desempate	% dif. promedio	Dif. promedio	Exactas
$\frac{tr}{p}$	$ts - tr$	1.97 %	66.94	219/5000
$\frac{tr}{p}$	supervivencia	1.98 %	67.12	215/5000
$\frac{tr}{p}$	día perfecto	2.08 %	70.56	168/5000
$\frac{tr}{p}$	prioridad alta	2.20 %	74.74	143/5000
supervivencia	$\frac{tr}{p}$	26.14 %	924.86	0/5000
supervivencia	prioridad alta	27.65 %	978.98	0/5000
supervivencia	día perfecto	31.47 %	1115.38	0/5000
supervivencia	$ts - tr$	32.48 %	1150.57	0/5000
prioridad alta	$\frac{tr}{p}$	43.93 %	1712.93	0/5000
prioridad alta	supervivencia	48.67 %	1882.31	0/5000
prioridad alta	$ts - tr$	60.13 %	2315.99	0/5000
prioridad alta	día perfecto	63.64 %	2455.57	0/5000
$ts - tr$	$\frac{tr}{p}$	69.78 %	2517.59	0/5000
$ts - tr$	supervivencia	70.82 %	2556.30	0/5000
$ts - tr$	prioridad alta	71.65 %	2587.62	0/5000
$ts - tr$	día perfecto	74.73 %	2699.56	0/5000
día perfecto	$\frac{tr}{p}$	77.50 %	2810.43	0/5000
día perfecto	supervivencia	80.84 %	2931.55	0/5000
día perfecto	prioridad alta	83.28 %	3034.64	0/5000
día perfecto	$ts - tr$	90.03 %	3270.25	0/5000

Resumen estadístico descriptivo. El ranking muestra una separación muy marcada entre familias de criterios. Las cuatro variantes con $\frac{tr}{p}$ como criterio principal ocupan los cuatro primeros lugares, con exceso relativo promedio entre 1.97 % y 2.20 %, diferencia promedio entre 66.94 y 74.74, y entre 143 y 219 coincidencias exactas en costo con el óptimo. La mejor combinación fue $\frac{tr}{p}$ con desempate por $ts - tr$, seguida muy de cerca por $\frac{tr}{p}$ con supervivencia. En cambio, la mejor variante que ya no usa $\frac{tr}{p}$ como criterio principal salta a 26.14 % de exceso promedio y 0 coincidencias exactas en costo, mientras que el peor caso del barrido fue día perfecto con desempate por $ts - tr$, con 90.03 %.

Interpretación de la comparación de criterios. La conclusión es que el desempate sí influye, pero su efecto es secundario frente a la elección del criterio principal. Lo determinante es conservar $\frac{tr}{p}$ como clave base, porque apenas se reemplaza por urgencia, prioridad, supervivencia o día perfecto, el rendimiento cae de forma abrupta. Dentro del grupo correcto, el mejor desempate sigue siendo la holgura $ts - tr$, por lo que esa combinación queda empíricamente justificada como la versión más robusta del criterio voraz implementado.

4.2.3. Síntesis e interpretación de la evidencia

Análisis de la demostración formal y de los resultados de simulación. Teoría y evidencia apuntan a la misma idea:

- el problema general no cumple la propiedad de escogencia voraz, porque la función de costo mezcla tres regímenes;
- en los casos 1 y 2 a veces conviene esperar para aprovechar rp o conservar margen;
- en el caso 3 retrasarse es caro y además el daño se multiplica por la prioridad;
- por eso no existe una regla local que sea siempre exacta;
- aun así, el argumento de intercambio de la Sección 4.1 sí justifica $\frac{tr}{p}$ cuando la instancia está dominada por tardanzas.

Síntesis de lo encontrado en los experimentos. Los experimentos refuerzan esa lectura:

- **Batería de pruebas + contraejemplo formal:** el voraz sí puede fallar; en 18 casos solo alcanzó una vez el costo óptimo y el exceso relativo promedio fue 16.23 %.
- **Bloque mixto de 5000 fincas:** la tasa de costo óptimo fue baja ($219/5000 = 4,38\%$), pero el exceso relativo promedio bajó a 1.97 %.
- **Perfiles puros:** fue perfecto en crítico, fuerte en urgente y perdió precisión al alcanzar el costo óptimo cuando aumentó la holgura, sobre todo en relajado.
- **Comparación de 20 combinaciones:** las cuatro mejores variantes mantienen $\frac{tr}{p}$ como criterio principal; la mejor que lo abandona ya sube a 26.14 % de exceso promedio.

Cuándo acierta y cuándo falla según la estructura de costos. Con base en esa evidencia conjunta, el criterio tiende a acertar cuando la instancia está dominada por el costo de llegar tarde y falla cuando la decisión depende más de administrar margen y de coordinar el día perfecto rp :

- **Acierta mejor cuando casi todo el riesgo está en entrar al caso 3.**

Eso ocurre en perfiles críticos y urgentes, donde el margen $ts - tr$ es nulo o pequeño. Si esperar casi siempre empeora el costo, la regla $\frac{tr}{p}$ se alinea bien con la presión real del problema.

- *Ejemplo:* si un tablón tiene $margen = 0$, cualquier espera adicional ya lo hace llegar tarde; si otro además tiene tr pequeño y prioridad alta, regarlo primero reduce rápido una penalización potencial grande.

- **Puede elegir un orden no óptimo sin alejarse mucho del costo óptimo.**

En mezclas realistas, el voraz puede ordenar algunos tablonos de forma distinta a un orden de costo óptimo y aun así terminar cerca del mejor costo posible. Eso explica por qué en el bloque mixto la tasa de costo óptimo fue baja, pero el exceso relativo promedio siguió siendo solo 1.97 %.

- *Ejemplo:* si dos tablonos están ambos cerca del caso 3 y sus razones $\frac{tr}{p}$ son muy similares, cambiar su orden puede impedir alcanzar exactamente el costo óptimo sin producir una diferencia grande en el costo total.

- **Falla más cuando el problema consiste en decidir quién puede esperar sin daño.**

Eso aparece en perfiles balanceados, largo plazo y sobre todo relajados, donde el margen es amplio. Allí ya no basta con evitar tardanzas; hay que decidir si conviene usar el tiempo actual o reservarlo para alinear mejor otro tablón con su día perfecto rp .

- *Ejemplo:* dos tablonos pueden seguir en caso 2 aunque esperen, pero uno tiene $rp = 2$ y el otro $rp = 8$; el mejor orden puede depender de aprovechar ese rp temprano, no de la razón $\frac{tr}{p}$.

- **Falla especialmente cuando una decisión local desplaza a un tablón más sensible.**

Si un tablón posponible se atiende primero porque tiene buena razón $\frac{tr}{p}$, puede consumir tiempo y empujar a otro tablón hacia un peor régimen de costo. El problema no es solo evaluar mal un tablón aislado, sino no anticipar su efecto sobre los siguientes.

- *Ejemplo:* un tablón relajado puede esperar varios días sin problema, pero si se riega antes que uno urgente con $margen = 1$, ese segundo tablón puede pasar de caso 2 a caso 3 y encarecer mucho la solución.

Resumen. En relación con el problema original, la evidencia unificada permite afirmar lo siguiente:

- $\frac{tr}{p}$ no resuelve exactamente el problema general, porque la corrección depende de alcanzar un costo óptimo y ese resultado no se determina solo por urgencia y prioridad, sino también por cuándo conviene esperar.
- Aun así, $\frac{tr}{p}$ es la mejor regla local evaluada en el proyecto: fue la base de las cuatro mejores combinaciones del laboratorio comparativo y domina claramente a priorizar supervivencia, prioridad alta, margen o día perfecto como criterio principal.
- Por tanto, el voraz debe entenderse como una heurística rápida y bien fundada para aproximar el costo óptimo, especialmente útil cuando se necesita respuesta inmediata; si se requiere garantizar una solución de costo óptimo para el caso general, hace falta un método global como programación dinámica.

4.3. Complejidad y Corrección

Complejidad. El algoritmo voraz ejecuta dos fases:

1. ordena los n tableros según el criterio $\frac{tr}{p}$ con desempate por holgura;
2. evalúa la permutación resultante.

El siguiente pseudocódigo resume esa estructura:

```
Funcion roV(tableros)
    orden <- [0,1,...,n-1]
    ordenar(orden, comparar)
    retornar evaluar_orden(tableros, orden)
Funcion comparar(i, j)
    si  $tr_i p_j \neq tr_j p_i$  retornar  $tr_i p_j < tr_j p_i$ 
    si  $ts_i - tr_i \neq ts_j - tr_j$  retornar  $ts_i - tr_i < ts_j - tr_j$ 
    retornar  $i < j$ 
```

Proposición. En el pseudocódigo anterior, el paso `ordenar(orden, comparar)` cuesta $\Theta(n \log n)$ en la implementación del proyecto.

Demostración. Sea n el número de tableros. La inicialización

$$\text{orden} \leftarrow [0, 1, \dots, n - 1]$$

cuesta $O(n)$. El paso dominante del pseudocódigo es la operación de ordenamiento. En la notación del pseudocódigo, esa operación corresponde a `ordenar`. Como el comparador realiza un número constante de operaciones aritméticas y comparaciones enteras, cada comparación cuesta $O(1)$. Por tanto, para la cota superior se obtiene¹

$$T_{\text{ordenar}}(n) = O(n \log n).$$

Para la cota inferior, basta observar que el paso abstracto `ordenar(orden, comparar)` pertenece a la familia de ordenamientos por comparaciones, porque decide el orden relativo usando exclusivamente el comparador `comparar`. Ordenar n elementos distintos en ese modelo exige distinguir entre las $n!$ permutaciones posibles. Cada comparación solo tiene dos resultados, así que después de h comparaciones solo pueden diferenciarse a lo sumo 2^h casos. Por lo tanto, necesariamente debe cumplirse

$$2^h \geq n!,$$

¹En la implementación concreta del proyecto, el paso `ordenar(orden, comparar)` se resuelve con `std::sort`. Véase el borrador del estándar C++, sección [\[sort\]](#), donde se especifica la complejidad de `std::sort` como $O(N \log N)$ comparaciones y proyecciones.

y al tomar logaritmos se obtiene

$$h \geq \log_2(n!).$$

Ahora bien, en el producto $n! = 1 \cdot 2 \cdot \dots \cdot n$, los últimos $n/2$ factores son al menos $n/2$. Entonces

$$n! \geq \left(\frac{n}{2}\right)^{n/2},$$

y por tanto

$$h \geq \log_2(n!) \geq \frac{n}{2} \log_2\left(\frac{n}{2}\right) = \Omega(n \log n).$$

En consecuencia, cualquier ordenamiento por comparaciones requiere $\Omega(n \log n)$ comparaciones en el peor caso. Como ya se tenía la cota superior $O(n \log n)$, se concluye que

$$T_{\text{ordenar}}(n) = \Theta(n \log n).$$

Como la evaluación posterior de la permutación recorre los n tableros una sola vez, su costo es $O(n)$ y no modifica el orden asintótico total de **roV**. Por tanto, la complejidad temporal total del pseudocódigo, y de la implementación que lo materializa, es

$$\Theta(n \log n).$$

El espacio auxiliar es $O(n)$, debido al arreglo de índices que almacena la permutación resultante.

Correctitud del algoritmo voraz. El algoritmo voraz no siempre da la respuesta correcta al problema, entendiendo por respuesta correcta una programación de riego con costo óptimo. Aunque el problema sí tiene subestructura óptima, el criterio local usado por el algoritmo no garantiza la propiedad de escogencia voraz. Por esta razón, escoger en cada paso el tablón que parece más conveniente según $\frac{tr}{p}$ no asegura necesariamente obtener la mejor solución global.

En consecuencia, el algoritmo debe entenderse como una heurística eficiente: puede coincidir con el óptimo en ciertos tipos de instancias, especialmente cuando el retraso domina el costo, pero no es exacto para el caso general. A continuación se justifica formalmente esta afirmación mediante la subestructura óptima del problema y un contraejemplo a la escogencia voraz.

Subestructura óptima. Sí se cumple. Sea

$$\Pi^* = \langle \pi_0^*, \pi_1^*, \dots, \pi_{n-1}^* \rangle$$

una programación óptima. Si fijamos el primer tablón π_0^* , entonces el resto de la programación siempre comienza después de $tr_{\pi_0^*}$. Por tanto, el sufijo

$$\langle \pi_1^*, \dots, \pi_{n-1}^* \rangle$$

debe ser óptimo para ese subproblema residual. Si no lo fuera, existiría otro orden sobre esos mismos tableros con menor costo; al reemplazar solo el sufijo en Π^* , se obtendría una programación total más barata, contradicción.

Por ejemplo, si una solución óptima fuera $\langle 2, 0, 1, 3 \rangle$, entonces el sufijo $\langle 0, 1, 3 \rangle$ debe ser el mejor posible una vez regado T_2 . Si existiera un sufijo mejor, como $\langle 1, 0, 3 \rangle$, entonces $\langle 2, 1, 0, 3 \rangle$ costaría menos que la supuesta solución óptima. Por lo tanto, el problema sí tiene subestructura óptima.

Propiedad de escogencia voraz. No se cumple en general. Considérese la finca con dos tableros

$$T_0 = \langle 10, 2, 1, 0 \rangle, \quad T_1 = \langle 5, 3, 4, 0 \rangle.$$

Como

$$\frac{tr_0}{p_0} = 2,00, \quad \frac{tr_1}{p_1} = 0,75,$$

el criterio voraz elige primero a T_1 . Sin embargo:

Permutación	Cálculo	Costo total
$\langle 1, 0 \rangle$	$T_1 : 5 - (0 + 3) = 2, T_0 : 2(10 - (3 + 2)) = 10$	12
$\langle 0, 1 \rangle$	$T_0 : 10 - (0 + 2) = 8, T_1 : 2(5 - (2 + 3)) = 0$	8

Tabla 10: Contraejemplo a la propiedad de escogencia voraz.

La solución óptima comienza por T_0 , no por el tablero de menor $\frac{tr}{p}$. Esto demuestra formalmente que no existe garantía de que una solución óptima empiece con la escogencia voraz.

5. Solución Mediante Programación Dinámica

5.1. Descripción de la Estrategia

La implementación de programación dinámica del proyecto corresponde a la función `roPD`. El algoritmo recibe una finca

$$F = \langle T_0, \dots, T_{n-1} \rangle$$

y determina la permutación de riego de costo mínimo explorando todos los subconjuntos posibles de tableros mediante una máscara de bits.

Cada estado de la DP está representado por un entero $M \in \{0, \dots, 2^n - 1\}$, donde el bit i indica si el tablero i ya fue regado. Para cada estado se guarda el costo mínimo acumulado hasta llegar a él y el tiempo total consumido, definido como

$$\tau(M) = \sum_{\substack{i=0 \\ \text{bit } i \in M}}^{n-1} tr_i.$$

A partir del estado M , se itera sobre todos los tableros $i \notin M$ y se intenta agregarlos al siguiente lugar de la secuencia. Si el nuevo costo es menor que el conocido para el estado $M \cup \{i\}$, ese valor se actualiza junto con la información de reconstrucción. De esta forma se asegura que, al finalizar el recorrido de todos los estados, el que represente el conjunto completo de tableros $\{0, \dots, n-1\}$ contenga el costo óptimo global.

Pseudocódigo.

```

Funcion roPD(tablones):
  Para todo M en  $\{0, \dots, 2^n - 1\}$ :
    DP[M] <- INF, tiempo[M] <- 0
  DP[0] <- 0
  Para cada M de 0 hasta  $2^n - 1$ :
    Si DP[M] = INF: continuar
    Para cada tablon i no en M:
      t <- tiempo[M]
      costo_i <- calcularCosto(tablones[i], t)
      nuevo_costo <- DP[M] + costo_i
      M' <- M union {i}
      Si nuevo_costo < DP[M']:
        DP[M'] <- nuevo_costo
        tiempo[M'] <- t + tablones[i].tr
        padre[M'] <- M, ultimo[M'] <- i
  Reconstruir permutacion optima desde estado completo
  Retornar permutacion y DP[estado completo]

```

5.1.1. Correspondencia entre estrategia e implementación

El siguiente fragmento de la implementación en C++ muestra el núcleo del algoritmo:

```

for (int mask = 0; mask < total_states; ++mask) {
  if (dp[mask] == INF) continue;
  for (int i = 0; i < n; ++i) {
    if (mask & (1 << i)) continue;
    int t = time_used[mask];
    long long cost_i = calcularCosto(tablones[i], t);
    int next_mask = mask | (1 << i);
    if (dp[mask] + cost_i < dp[next_mask]) {
      dp[next_mask] = dp[mask] + cost_i;
      time_used[next_mask] = t + tablones[i].tiempo_riego;
      parent[next_mask] = mask;
      last_added[next_mask] = i;
    }
  }
}

```

Brevemente, cada elemento del fragmento cumple su rol dentro de la estrategia:

- `mask` recorre los 2^n subconjuntos posibles de tablones en orden creciente.
- `dp[mask] == INF` evita procesar estados que todavía no han sido alcanzados.
- `mask & (1 << i)` verifica si el tablón i ya fue incluido en el subconjunto actual.
- `time_used[mask]` recupera el tiempo acumulado $\tau(M)$ para calcular el instante de inicio del siguiente tablón.

- `calcularCosto(tablones[i], t)` evalúa la función de costo $c_i(t)$ para el tablón i comenzando en el instante t .
- `next_mask = mask | (1 << i)` construye el nuevo estado al agregar el tablón i .
- La condición de mejora actualiza el costo mínimo, el tiempo acumulado y la información de reconstrucción (`parent` y `last_added`).

5.2. Subestructura Óptima y Corrección

5.2.1. Subestructura óptima

Sea $OPT(S)$ el costo mínimo de regar exactamente los tablones de S en los primeros $|S|$ lugares de la programación. Si una programación óptima para S termina con el tablón $i \in S$, entonces el prefijo formado por $S \setminus \{i\}$ debe ser una solución óptima para ese subconjunto. Si no lo fuera, podría reemplazarse por uno de menor costo, obteniendo una mejor programación para S , en contradicción con la suposición de optimalidad.

Por eso cada subproblema queda bien definido por su subconjunto S , y el número total de subproblemas distintos es 2^n .

5.2.2. Definición recursiva

Para cada tablón i , la función de costo local en función del instante de inicio t es:

$$c_i(t) = \begin{cases} ts_i - (t + tr_i), & \text{si } t = rp_i, \\ 2(ts_i - (t + tr_i)), & \text{si } t \leq ts_i - tr_i \text{ y } t \neq rp_i, \\ 2p_i((t + tr_i) - ts_i), & \text{en otro caso.} \end{cases}$$

La recurrencia, agregando el último tablón del subconjunto, es:

$$OPT(\emptyset) = 0, \quad OPT(S) = \min_{i \in S} \{OPT(S \setminus \{i\}) + c_i(\tau(S) - tr_i)\}.$$

La implementación recorre los estados en sentido hacia adelante: desde el estado M añade el tablón $i \notin M$ y actualiza el estado $M \cup \{i\}$:

$$DP[M \cup \{i\}] = \min(DP[M \cup \{i\}], DP[M] + c_i(\tau(M))).$$

5.2.3. Corrección

La corrección del algoritmo se apoya directamente en la subestructura óptima. Como el recorrido progresa desde el estado vacío hasta el estado completo procesando todos los estados en orden creciente, cuando se evalúa la transición $M \rightarrow M \cup \{i\}$ el valor $DP[M]$ ya es el costo mínimo de regar exactamente los tablones de M . Por inducción sobre el tamaño del subconjunto, al finalizar el recorrido, $DP[\text{estado completo}]$ contiene el costo mínimo global.

A diferencia del enfoque voraz, la programación dinámica no toma decisiones locales: considera todas las transiciones posibles y conserva siempre la mejor. Por eso garantiza encontrar la solución óptima.

5.3. Aplicación del algoritmo y evaluación experimental

5.3.1. Aplicación a los ejemplos del enunciado

Ejemplo 1. Para la finca

$$F_1 = \langle \langle 10, 3, 4, 0 \rangle, \langle 6, 3, 3, 1 \rangle, \langle 2, 2, 1, 0 \rangle, \langle 8, 1, 1, 6 \rangle, \langle 10, 4, 2, 5 \rangle \rangle$$

la programación dinámica produce la permutación óptima:

$$\Pi^* = \langle 2, 1, 0, 3, 4 \rangle.$$

Paso	Tablón	t_{inicio}	Caso aplicado	Costo
1	T_2	0	caso 1: $2 - (0 + 2)$	0
2	T_1	2	caso 2: $2(6 - (2 + 3))$	2
3	T_0	5	caso 2: $2(10 - (5 + 3))$	4
4	T_3	8	caso 3: $2 \cdot 1 \cdot ((8 + 1) - 8)$	2
5	T_4	9	caso 2: $2(10 - (9 + 4))$	...

Tabla 11: Evaluación detallada de la PD sobre F_1 .

$$CR_{F_1}^{\Pi^*} = 20.$$

Frente al costo de 34 obtenido por el voraz, la PD reduce el costo en 14 unidades al evitar que T_2 (con holgura cero) quede postergado.

Ejemplo 2. Para la finca

$$F_2 = \langle \langle 9, 3, 4, 0 \rangle, \langle 5, 3, 3, 2 \rangle, \langle 2, 2, 1, 0 \rangle, \langle 8, 1, 1, 6 \rangle, \langle 6, 4, 2, 1 \rangle \rangle$$

la PD produce igualmente

$$\Pi^* = \langle 2, 1, 0, 3, 4 \rangle \quad \text{con costo} \quad 32,$$

frente a los 55 del voraz. El patrón de mejora es el mismo: atender primero T_2 (margen nulo) evita que ingrese al caso 3.

5.3.2. Evaluación experimental

Para los 18 casos de la batería del repositorio que cuentan con salida de referencia, la programación dinámica coincidió exactamente con el costo óptimo en todos los casos. Estos resultados sirvieron, además, como referencia para cuantificar las desviaciones del algoritmo voraz reportadas en la sección anterior.

Para instancias más grandes, los tiempos de ejecución crecen de acuerdo con la complejidad exponencial del algoritmo. Los experimentos del bloque mixto de $n = 20$ se resolvieron todos por PD en tiempos del orden de segundos, pero instancias con $n = 32$ ya dejan de ser viables, como se detalla en la siguiente subsección.

5.4. Análisis de Complejidad

Complejidad temporal. La programación dinámica recorre los 2^n estados y, desde cada uno, intenta agregar cada uno de los n tableros aún no regados. Por ello, el número total de transiciones es del orden de $n2^n$, y la complejidad temporal es:

$$O(n2^n).$$

Complejidad espacial. El espacio está dominado por los arreglos indexados por máscara, cada uno de tamaño 2^n . La implementación mantiene tablas para costo (`dp`), tiempo acumulado (`time_used`), estado padre (`parent`) y último tablero agregado (`last_added`). Eso aumenta el factor constante, pero no cambia el orden asintótico:

$$O(2^n).$$

Utilidad práctica. Bajo la hipótesis del enunciado de 3×10^8 operaciones por minuto y 4 bytes por celda, se obtiene el siguiente panorama para $n = 2^k$:

k	$n = 2^k$	$n2^n$ operaciones	Tiempo aproximado	Memoria base $4 \cdot 2^n$
3	8	2,048	despreciable	1 KB
4	16	1,048,576	0.0035 min	256 KB
5	32	137,438,953,472	458 min	16 GB

Tabla 12: Estimación práctica de la programación dinámica.

La programación dinámica es perfectamente viable para tamaños pequeños y medianos, pero deja de ser práctica cuando n supera los 20 tableros aproximadamente. A partir de $n = 32$ el tiempo y la memoria crecen demasiado rápido. Aun así, constituye una mejora enorme frente a la fuerza bruta, cuyo crecimiento factorial la vuelve inviable mucho antes: mientras la fuerza bruta no escala más allá de $n \approx 10$, la PD alcanza cómodamente $n = 20$ y en la práctica del proyecto resolvió todas las instancias de la batería con exactitud garantizada.

6. Comparación de Estrategias

La Tabla 13 resume las principales diferencias entre los tres enfoques desarrollados.

Desde el punto de vista de calidad de solución, fuerza bruta y programación dinámica son los enfoques correctos para el problema general. La diferencia entre ellos está en la eficiencia: la programación dinámica evita recalcular subproblemas equivalentes y por eso reemplaza el crecimiento factorial por uno exponencial de base 2.

Estrategia	Complejidad temporal	Garantía de optimalidad	Uso práctico
Fuerza bruta	$O(n!n)$	Sí	Solo instancias muy pequeñas; sirve como referencia exacta.
Voraz	$O(n \log n)$	No	Muy rápido; útil para obtener una respuesta inmediata.
Programación dinámica	$O(n2^n)$	Sí	Exacta y mucho más escalable que fuerza bruta para instancias moderadas.

Tabla 13: Comparación general de las estrategias.

Desde el punto de vista de tiempo de respuesta, el voraz es claramente superior. Su costo $O(n \log n)$ lo hace apropiado para interfaces interactivas o escenarios en los que se necesite una estimación rápida. No obstante, los resultados observados en la batería del proyecto muestran que esta ganancia en velocidad tiene un precio importante en calidad: el voraz no captura adecuadamente la interacción entre prioridad, margen de supervivencia y día perfecto.

En consecuencia, una arquitectura razonable para el proyecto es usar:

- fuerza bruta como validador para instancias pequeñas;
- programación dinámica como método exacto principal;
- voraz como alternativa rápida cuando el tamaño de entrada o el contexto de uso desaconsejan un algoritmo exacto.

7. Conclusiones

El problema del riego óptimo no puede resolverse correctamente con una intuición local simple, porque el costo de cada tablón depende del instante exacto en que comienza su riego y ese instante, a su vez, depende de todas las decisiones previas. Esa dependencia global explica por qué el enfoque voraz, aunque eficiente, no siempre logra soluciones óptimas.

La fuerza bruta ofrece una solución conceptualmente sencilla y totalmente correcta, pero su complejidad factorial la limita a instancias muy pequeñas. La programación dinámica, en cambio, aprovecha la subestructura óptima del problema y permite obtener soluciones exactas con una mejora sustancial en eficiencia. Aun así, su crecimiento exponencial impone un límite práctico para valores grandes de n .

En este proyecto, la mejor relación entre exactitud y escalabilidad la ofrece la programación dinámica. El algoritmo voraz resulta valioso como heurística y como punto de comparación experimental, pero no reemplaza un método exacto cuando se requiere garantizar optimalidad. La comparación entre los tres enfoques muestra precisamente la idea central del curso: distintas estrategias de diseño algorítmico pueden modelar el mismo problema, pero sus beneficios y limitaciones son profundamente diferentes.